CYBER SECURITY INTERNSHIP | WEEK 9

SOC Introduction & Lab Setup

Security Operations Center fundamentals and a hands-on IDS monitoring environment on Ubuntu Server

Submitted by: [Your Name]

Module: Security Operations Center (SOC)

Date: 5 November 2026

Contents

1. Objective and Scope 3

Work completed 3

Tools used (all free) 3

Lab architecture 4

2. Task 1 - SOC Basics 4

2.1 What is a Security Operations Center? 4

2.2 Analyst Roles: L1, L2 and L3 5

2.3 SIEM vs IDS vs IPS 6

3. Task 2 - Environment Setup 7

3.1 Virtual machine and operating system 7

3.2 System update 7

3.3 Hostname setup 8

3.4 Network configuration 8

4. Task 3 - Installing the IDS 9

4.1 Attempting to install Snort 9

4.2 Choosing Suricata as the alternative 10

4.3 Verifying the installation 11

4.4 Running in test mode 12

5. Task 4 - Creating Rules 13

5.1 Understanding rule structure 13

5.2 Network variables and rule loading 13

5.3 The two custom rules 14

5.4 Fixing a formatting mistake 14

6. Task 5 - Testing and Alert Capture 15

6.1 Connectivity check 15

6.2 Starting Suricata and generating ICMP traffic 15

6.3 Reading the alerts 16

6.4 HTTP traffic test 17

7. Analysis and Observations 18

7.1 Summary of results 18

7.2 Problems met and how they were solved 18

7.3 How a SOC analyst would use these alerts 18

7.4 Improving the rules 18

8. Conclusion 19

Key lessons 19

Future work 19

Appendix A - Commands used 20

Appendix B - References 20

1. Objective and Scope

This report documents Week 9 of the internship. The goal is twofold: first, to understand how a Security Operations Center (SOC) works, and second, to build a small but real monitoring environment where network traffic is inspected by an Intrusion Detection System (IDS) and alerts can be reviewed the way a SOC analyst would.

Work completed

- **Task 1:** SOC basics - what a SOC is, analyst roles (L1/L2/L3) and the difference between SIEM, IDS and IPS.
- **Task 2:** Environment - VirtualBox virtual machine running Ubuntu Server, system update, hostname and network checks.
- **Task 3:** IDS installation, verification and configuration test (Suricata, after Snort proved unavailable).
- **Task 4:** Understanding rule structure and writing two custom rules: ICMP ping detection and HTTP traffic detection.
- **Task 5:** Generating traffic (ping and web) and capturing the alerts, with screenshots as evidence.

Note on the IDS tool. The assignment names Snort. On this Ubuntu 26.04 LTS system the `snort` package is not available in the official repositories (see Section 4), so the same work was completed with **Suricata**, a free and open-source IDS that uses Snort-compatible rule syntax. Every step, rule and test in this report was performed on Suricata.

Tools used (all free)

Component	Details
Hypervisor	Oracle VirtualBox
Guest OS	Ubuntu Server 26.04.1 LTS (codename *resolute*)
VM / hostname	Ubuntu-SOC-Lab (user: vboxuser)
IDS engine	Suricata 8.0.3 RELEASE (installed from the Ubuntu repository)
Rules file	/etc/suricata/rules/local.rules
Alert log	/var/log/suricata/fast.log
Editor	GNU nano 8.7.1

Lab architecture

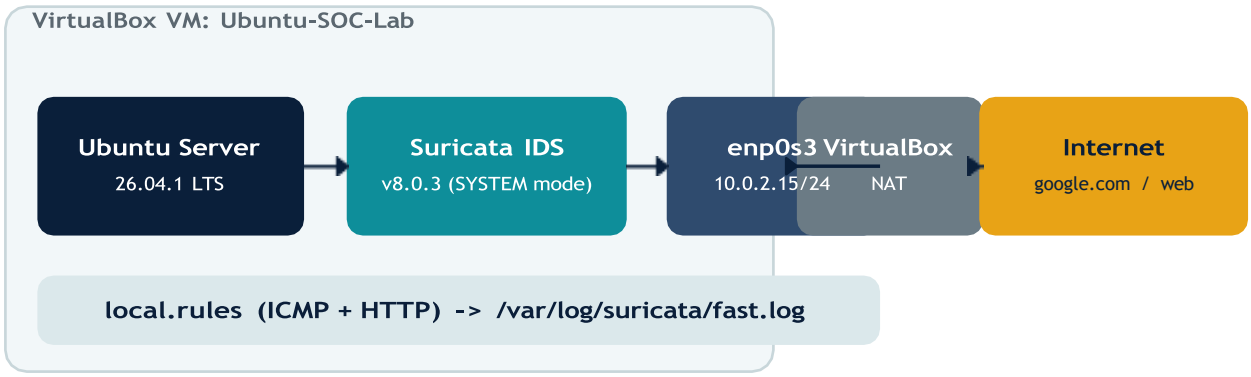


Figure 1. *Logical layout of the lab. Traffic leaving the VM through the enp0s3 interface is inspected by Suricata, which writes matching events to fast.log.*

2. Task 1 - SOC Basics

2.1 What is a Security Operations Center?

A Security Operations Center is the part of an organisation that watches over its digital systems around the clock. It brings together a trained team, defined procedures and monitoring technology so that suspicious activity is noticed quickly, judged correctly and handled before it turns into real damage. Rather than reacting after a breach is discovered by accident, a SOC aims to find problems while they are still small.

A useful way to remember how a SOC is built is the three pillars:

Pillar	What it means in a SOC
People	Analysts, incident responders, threat hunters, engineers and a SOC manager working in shifts to provide 24x7 coverage.
Process	Playbooks, escalation paths, ticketing and reporting. These keep every alert handled the same consistent way.
Technology	SIEM, IDS/IPS, endpoint tools, firewalls, threat-intelligence feeds and case-management platforms.

Core responsibilities

- **Continuous monitoring** of networks, servers, endpoints, cloud services and user activity.
- **Detection and triage:** separating real threats from false alarms and ranking them by severity.
- **Incident response:** containing the threat, removing it and restoring normal operations.
- **Log management:** collecting and retaining logs so that investigations and audits have evidence to work with.
- **Threat intelligence and hunting:** using knowledge of attacker behaviour to look for activity that tools missed.
- **Tuning and improvement:** refining detection rules so analysts spend time on genuine problems.

The incident handling cycle



Figure 2. *Typical flow of an incident through a SOC, from first alert to lessons learned.*

The first three stages are where most daily SOC work happens: an alert appears, an analyst triages it, and the suspicious event is analysed in more depth. Only confirmed incidents move on to containment and recovery. The final stage matters just as much, because every incident should leave the SOC with better rules, better playbooks or better awareness.

2.2 Analyst Roles: L1, L2 and L3

SOC work is organised in tiers. Work moves upward only when it needs more skill, more authority or more time than the current tier can give. This keeps the highest-skilled people free for the hardest problems.



Figure 3. *SOC tier model. Higher tiers handle fewer but more complex cases.*

	L1 - Triage Analyst	L2 - Incident Responder	L3 - Expert / Threat Hunter
Focus	First line of defence; watches the alert queue	Deep investigation of escalated incidents	Advanced threats and proactive defence
Typical tasks	Reviews alerts, checks if they are false positives, follows playbooks, opens tickets, escalates anything uncertain	Correlates logs from several sources, analyses suspicious files and traffic, performs containment (isolate host, block IP), advises on rule tuning	Hunts for hidden threats, performs forensics and malware analysis, builds new detections, leads major incidents, mentors the team
Tools	SIEM dashboard, ticketing system, IDS alerts	SIEM queries, EDR, packet captures, sandbox	Forensic suites, reverse-engineering tools, threat-intel platforms
Hands off to	L2	L3 or incident manager	Management, engineering, external partners

In smaller organisations the tiers blend together and one person may cover several of these duties. Other supporting roles often found around a SOC include the SOC manager, the detection engineer who writes and tunes rules, and the threat-intelligence analyst.

2.3 SIEM vs IDS vs IPS

These three technologies are often mentioned together because they cooperate, but they do different jobs.

- **SIEM (Security Information and Event Management)** gathers logs and events from many sources, normalises them, correlates them and raises alerts. It is the analyst's central workspace, not a traffic inspector.
- **IDS (Intrusion Detection System)** inspects network traffic or host activity and generates an alert when something matches a rule or looks abnormal. It only watches and reports; it does not block.
- **IPS (Intrusion Prevention System)** works like an IDS but is placed *inline*, so it can drop or reject malicious packets in real time.

Aspect	SIEM	IDS	IPS
Main purpose	Collect, correlate and analyse logs from everywhere	Detect suspicious traffic or activity	Detect and automatically stop malicious traffic
Data it sees	Logs and events from firewalls, servers, IDS, apps, endpoints	Packets on the network (NIDS) or activity on a host (HIDS)	Packets passing through it, inline
Action taken	Alerts, dashboards, reports, correlation	Alert only (passive)	Alert and block (active)
Placement	Central server receiving feeds	Out of band: mirror port, TAP or the host itself	Inline in the traffic path
Strength	Big-picture visibility and evidence retention	No risk of disrupting legitimate traffic	Stops known attacks immediately
Limitation	Only as good as the logs it receives; needs tuning	Cannot stop an attack by itself	A bad rule can block legitimate users
Examples	Splunk, Elastic Stack, Wazuh, IBM QRadar	Snort, Suricata, Zeek	Snort or Suricata in inline mode, commercial NGIPS

How they work together

In a typical SOC, an IDS on the network produces alerts such as the ones created in this lab. Those alerts, together with firewall and server logs, are forwarded to the SIEM. The SIEM correlates them, for example by noticing that the same IP triggered an IDS alert and then a failed-login burst, and presents a single prioritised case to the L1 analyst. If an IPS is deployed, it can additionally block the offending traffic automatically. The lab in this report covers the IDS layer of that chain.

Both IDS and IPS tools use two broad detection methods: **signature-based** (matching known patterns, as our custom rules do) and **anomaly-based** (flagging deviations from normal behaviour). Signature detection is precise for known threats but blind to new ones, while anomaly detection can spot the unknown at the cost of more false positives.

3. Task 2 - Environment Setup

3.1 Virtual machine and operating system

A virtual machine was created in Oracle VirtualBox, a free hypervisor, so the lab stays isolated from the host computer and can be rebuilt or snapshotted at any time. Ubuntu Server 26.04.1 LTS was installed into it, and the machine was named **Ubuntu-SOC-Lab**. Using a server edition keeps the system lightweight, since all work is done from the command line, which is how SOC tooling is usually managed.

The first boot output below shows cloud-init completing its final boot stage and generating the SSH host keys (RSA, ECDSA and ED25519) for `root@Ubuntu-SOC-Lab`, which confirms the installation finished and the system booted correctly.

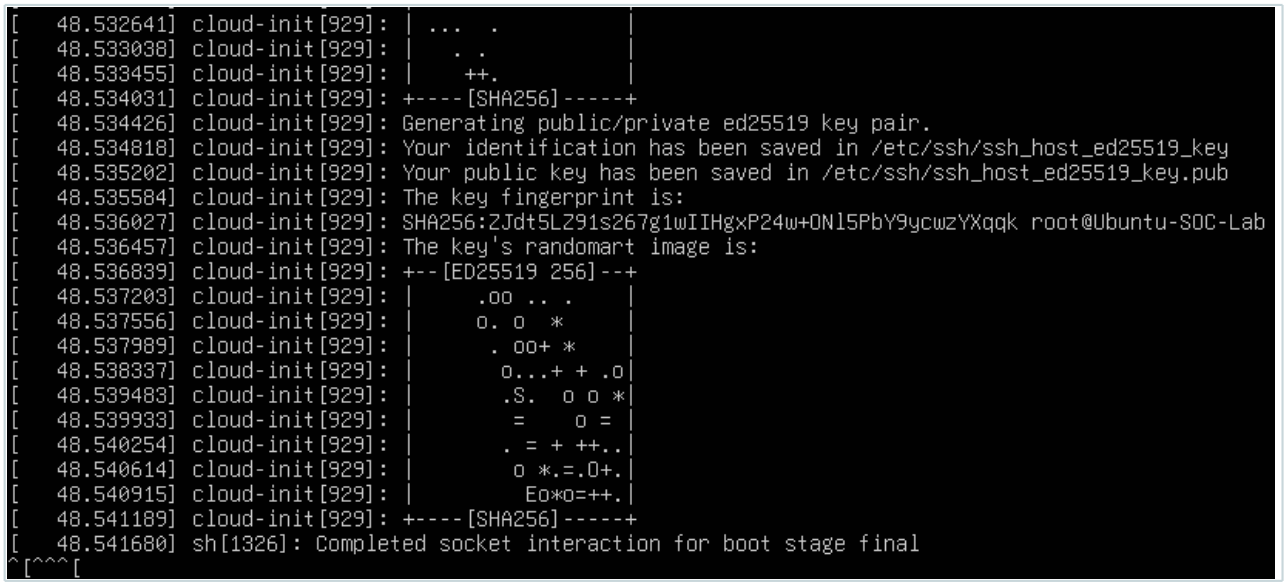


Figure 4. *First boot of the Ubuntu Server VM: SSH host keys generated for Ubuntu-SOC-Lab and boot stage completed.*

3.2 System update

Before installing any security tool, the package index was refreshed and installed packages were upgraded. Keeping the base system patched is the first rule of hardening, and it ensures newer packages (including the IDS later on) install without dependency conflicts.

```
sudo apt update
sudo apt upgrade -y
```

The screenshot below shows the upgrade downloading 55.6 MB of updates from the Ubuntu mirrors. When it finished, services were restarted and the running kernel was reported as up to date (visible in the next screenshot).



Figure 5. *System upgrade in progress: package downloads from the Ubuntu security and updates repositories.*

3.3 Hostname setup

The hostname identifies the machine in logs and on the network, so a meaningful name matters in a SOC where many hosts report to one place. The hostname was set to **Ubuntu-SOC-Lab** and verified with the `hostname` command; the shell prompt (`vboxuser@Ubuntu-SOC-Lab`) shows the same name.

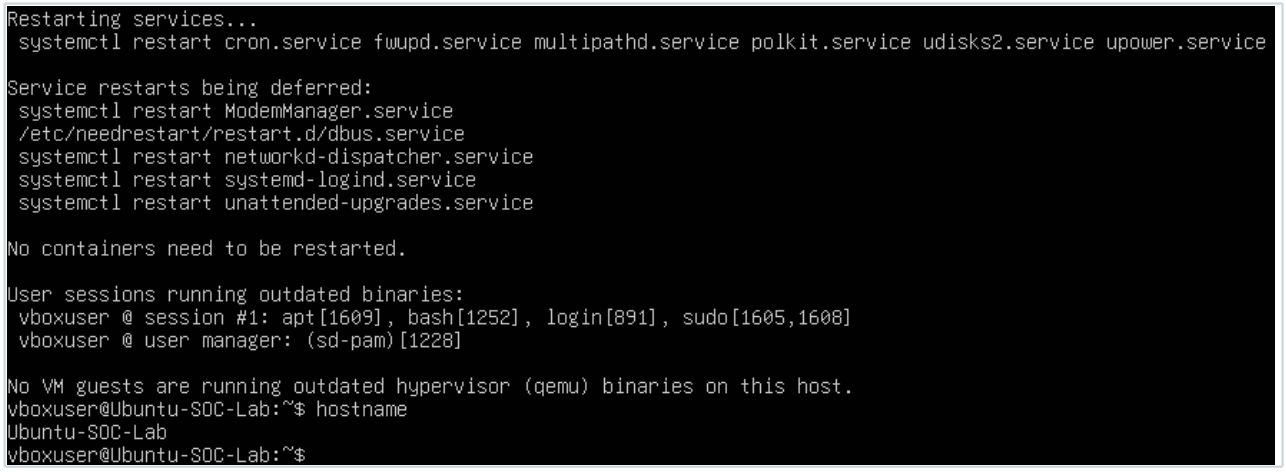


Figure 6. *Upgrade completed and hostname verified as Ubuntu-SOC-Lab.*

3.4 Network configuration

The network was checked with `ip a`. The interface **enp0s3** is UP and has received the address **10.0.2.15/24** by DHCP. This address range is the default one VirtualBox gives to a NAT adapter, so the VM can reach the internet through the host while remaining hidden from the outside network, which is a safe default for a lab.

Item	Value
Interface	enp0s3 (state UP)
IPv4 address	10.0.2.15/24, dynamic (DHCP)
Broadcast	10.0.2.255
MAC address	08:00:27:cb:a4:ac
IPv6	Link-local fe80:: address plus a global dynamic address
Loopback	lo, 127.0.0.1/8

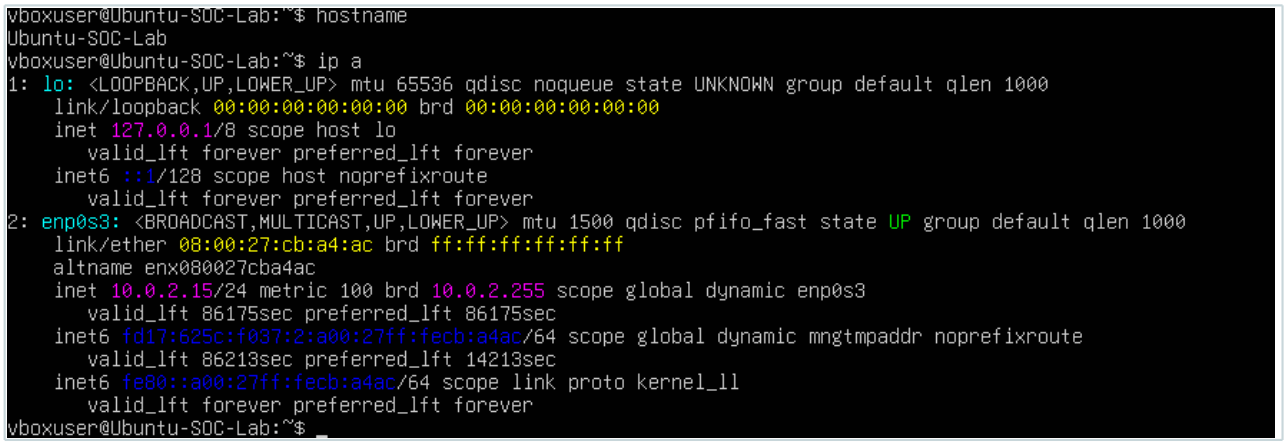


Figure 7. *Output of ip a showing enp0s3 with 10.0.2.15/24 and the loopback interface.*

4. Task 3 - Installing the IDS

4.1 Attempting to install Snort

The assignment asks for Snort, so the first attempt was a normal installation:

```
sudo apt install snort -y
```

The system answered `Error: Unable to locate package snort`. To make sure this was not a typing or configuration mistake, the following checks were done:

- `lsb_release -a` confirmed the release: Ubuntu 26.04.1 LTS, codename *resolute*.
- `sudo apt update` ran cleanly against the security, updates and backports repositories.
- `cat /etc/apt/sources.list.d/ubuntu.sources` showed that main, restricted, universe and multiverse are all enabled, so a missing component was not the cause.
- `apt search snort` listed only Snort-related helpers (such as libdaq, oinkmaster and psad) and no snort package itself.

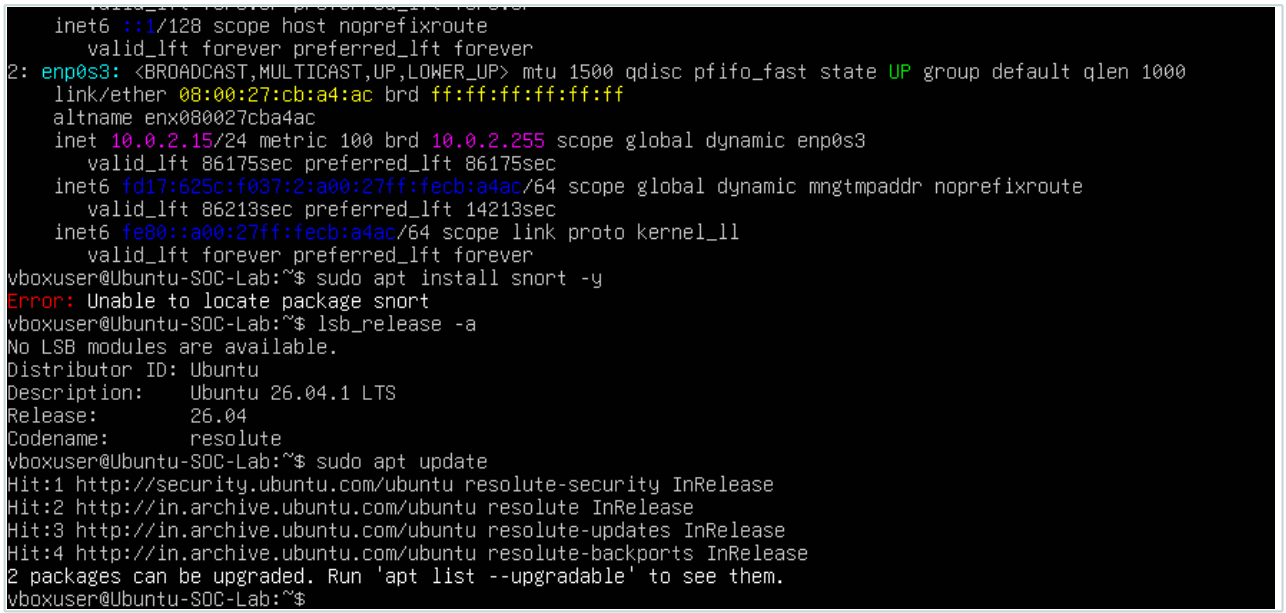


Figure 8. *Snort not found by apt; release checked with lsb_release and the package index refreshed.*

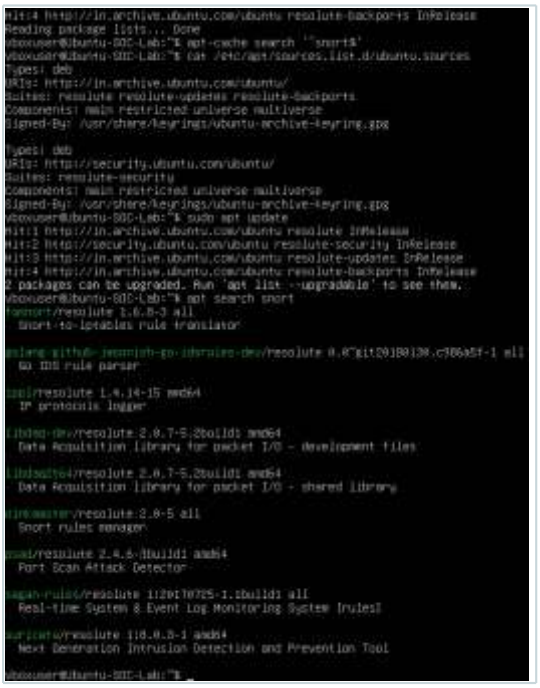


Figure 9. *Repository configuration and apt search snort. The result list includes suricata (Next Generation Intrusion Detection and Prevention Tool).*

4.2 Choosing Suricata as the alternative

Since Snort is not packaged for this release, an equivalent IDS was needed. The search output itself pointed to **Suricata**, which is a mature open-source engine maintained by the Open Information Security Foundation. It was chosen because:

- It is free and open source, which satisfies the 'only free tools' rule.
- It uses Snort-style rule syntax (action, protocol, addresses, direction and an option block), so the Task 4 rule-writing concepts are identical.
- It is a first-class IDS/IPS used in real SOCs, with built-in application-layer parsing for protocols such as HTTP, DNS and TLS.
- It is available directly from the Ubuntu repository as version 1:8.0.3-1, so no unofficial sources were required.

```
sudo apt install suricata -y
```

The installer pulled in Suricata and its libraries and created the `suricata.service` unit. During the post-install step the system printed *Could not execute systemctl* for the service start, which is a common message when the package is configured inside a freshly provisioned machine; the engine was later started manually and worked correctly (see Task 5).

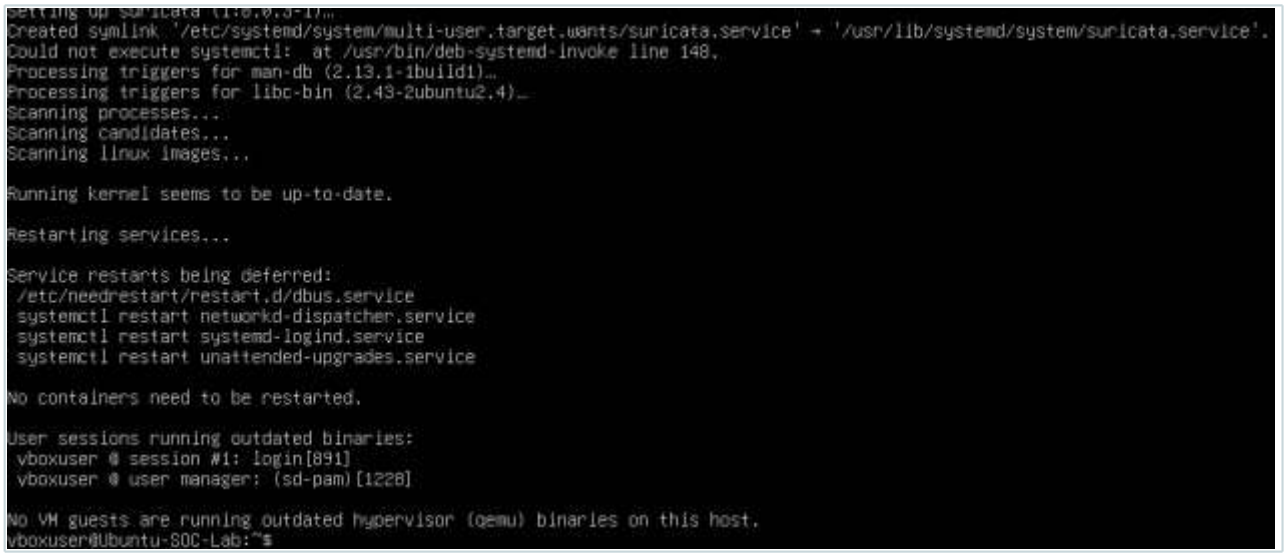


Figure 10. *Suricata 1:8.0.3-1 installed; the systemd service was linked into multi-user.target.*

4.3 Verifying the installation

Verification was done in two ways. The build information (`suricata --build-info`) confirms the version, the installation prefix `/usr`, the configuration directory `/etc/suricata/` and the log directory `/var/log/suricata/`. The help screen (`suricata -h`) lists the available run modes such as live capture (`-i`), offline pcap reading (`-r`) and the configuration test (`-T`).



Figure 11. *Suricata build information: install prefix, configuration directory and log directory.*



Figure 12. *Suricata help output listing run modes, detection options and info switches.*

4.4 Running in test mode

Test mode loads the configuration and rules, validates them and then exits without sniffing any traffic. It is the safest way to check that an IDS is healthy before running it on a network.

```
sudo suricata -T -c /etc/suricata/suricata.yaml
```

The first test run printed the version line (*Suricata version 8.0.3 RELEASE running in SYSTEM mode*) and a warning: `No rule files match the pattern /var/lib/suricata/rules/suricata.rules`. This meant the engine itself was fine, but no rules had been loaded yet because the default rule set is not fetched automatically. That warning led directly into Task 4, where custom rules were written. After the rules were in place, the same command finished with **Configuration provided was successfully loaded. Exiting.**, which is the success message for test mode.

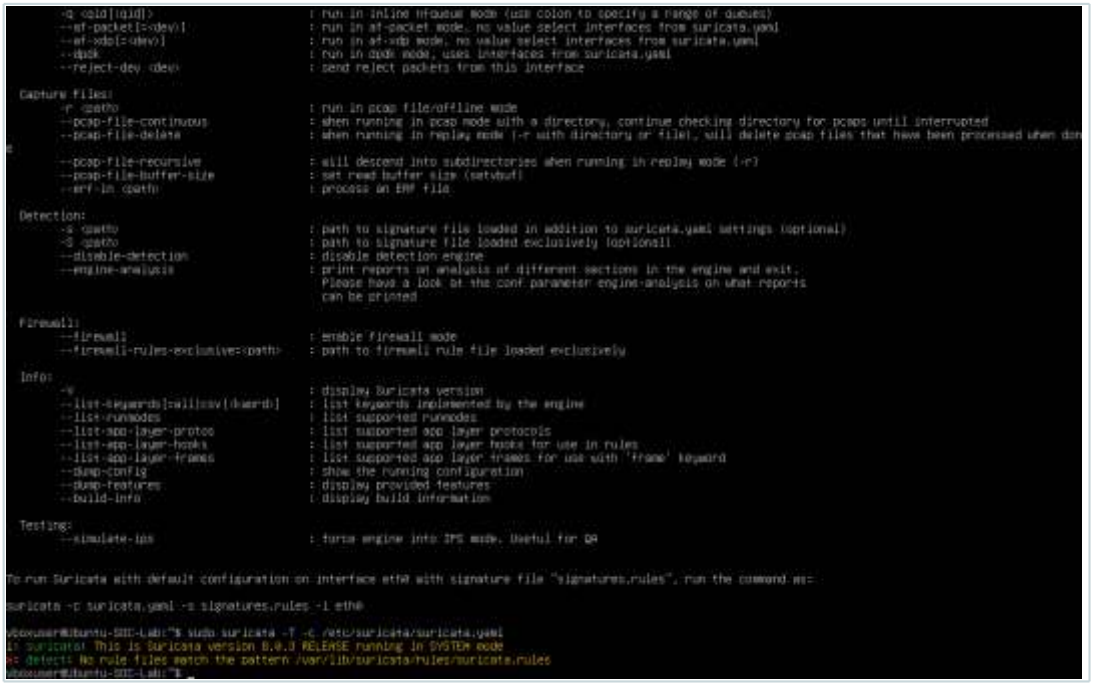


Figure 13. *First test-mode run: engine starts, but the warning shows that no rule files were found yet.*

5. Task 4 - Creating Rules

5.1 Understanding rule structure

Snort and Suricata rules share one format. Each rule is a single line made of a **header**, which decides which traffic to look at, and an **options block** in brackets, which describes what to match and how to report it.

alert	icmp	any	any	->	any	any	(msg:"..."; sid:...; rev:...;)
Action	Protocol	Source IP	Source port	Direction	Dest. IP	Dest. port	Options

Part	Meaning
Action	What to do on a match. `alert` writes an alert; other actions include drop and reject (IPS use).
Protocol	Traffic type to inspect, for example icmp, tcp, udp, or an application protocol such as http.
Addresses and ports	Source and destination. `any` matches everything; variables like $HOME_NET can narrow the scope.
Direction	`->` means source to destination. `<>` would match both directions.
msg	Human-readable text shown in the alert so an analyst knows what fired.
sid	Unique rule ID. Numbers from 1,000,000 upward are reserved for locally written rules.
rev	Revision number, increased every time the rule is edited.

5.2 Network variables and rule loading

Suricata's configuration file defines `HOME_NET`, the networks considered internal. In this lab it covers the private ranges `192.168.0.0/16`, `10.0.0.0/8` and `172.16.0.0/12`, so the VM address 10.0.2.15 is correctly treated as an internal host and `EXTERNAL_NET` (everything else) covers the internet. Further down, the `default-rule-path` and `rule-files` entries tell the engine where to find rules; this is the section that the earlier test-mode warning referred to.



Figure 14. *suricata.yaml, vars section: HOME_NET set to the three private ranges; EXTERNAL_NET defined as everything else.*



Figure 15. *suricata.yaml: default-rule-path and the rule-files list that controls which rule files are loaded.*

5.3 The two custom rules

Rules were written to `/etc/suricata/rules/local.rules`, the standard place for locally authored detections. The folder also contains the stock event rule files (for example http-events.rules, dns-events.rules and tls-events.rules) shipped with the package.

```
-rw-r--r-- 1 root root 15088 Jan  9  2026 http-events.rules
-rw-r--r-- 1 root root  3907 Jan  9  2026 http2-events.rules
-rw-r--r-- 1 root root  2834 Jan  9  2026 ipsec-events.rules
-rw-r--r-- 1 root root   585 Jan  9  2026 kerberos-events.rules
-rw-r--r-- 1 root root   155 Nov  3 18:54 local.rules
-rw-r--r-- 1 root root  2576 Jan  9  2026 mdns-events.rules
-rw-r--r-- 1 root root  1856 Jan  9  2026 modbus-events.rules
```

Figure 16. *Long listing of the rules directory: local.rules (155 bytes, created Nov 3) sits alongside the stock event rule files.*

```
alert icmp any any -> any any (msg:"ICMP Ping Detected"; sid:1000001; rev:1;)
alert http any any -> any any (msg:"HTTP Traffic Detected"; sid:1000002; rev:1;)
```

Rule	How it works
1000001 - ICMP Ping Detected	Matches any ICMP packet between any two hosts. When we ping a server, both the echo request (type 8) and the echo reply (type 0) are matched, so one ping produces two alerts.
1000002 - HTTP Traffic Detected	Uses the `http` application-layer protocol. Suricata decodes the traffic and fires when it recognises HTTP, regardless of which port is used. In classic Snort the equivalent would be a tcp rule on port 80.

Both rules use `any any -> any any` deliberately. For a first lab, matching everything guarantees the rule fires and proves the detection pipeline works from end to end. Section 7 discusses how a production rule would be narrowed.

5.4 Fixing a formatting mistake

When the file was first displayed with `cat`, the two rules appeared joined on a single line (*...rev:1;)alert http any any...*) because the newline between them was missing. Rules must each sit on their own line, so the file was reopened in nano, the rules were separated onto two lines, and the configuration was tested again.



Figure 17. *Contents of local.rules shown with cat: both rules are present but concatenated on one line.*



Figure 18. *local.rules reopened in nano: each rule now sits on its own line (Read 2 lines).*



Figure 19. *Configuration test after the rules were added: configuration successfully loaded.*

6. Task 5 - Testing and Alert Capture

6.1 Connectivity check

Before starting the IDS, outbound connectivity was confirmed with a ping to google.com. Three of four replies came back (25% loss, round-trip average about 92 ms), which is typical of a first request on a NAT interface warming up. The interface details were checked again with `ip a` and still showed 10.0.2.15 on enp0s3.

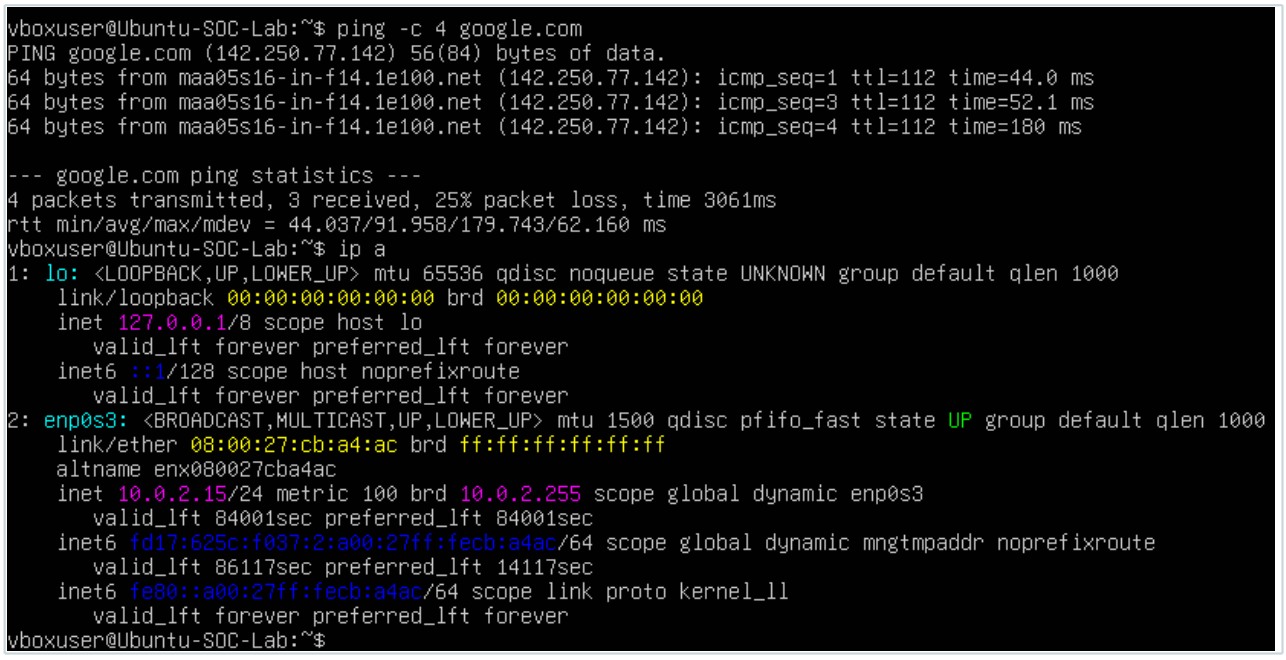


Figure 20. *Baseline ping to google.com before the IDS was started.*

6.2 Starting Suricata and generating ICMP traffic

Suricata was started as a background daemon, listening live on the VM's network interface with the configuration file loaded:

```
sudo suricata -D -c /etc/suricata/suricata.yaml -i enp0s3
```

A new ping test was then run (`ping -c 4 google.com`). This time all 4 packets were transmitted and received with 0% packet loss and an average round-trip time of about 50 ms. The alert log was read with:

```
sudo tail -n 20 /var/log/suricata/fast.log
```

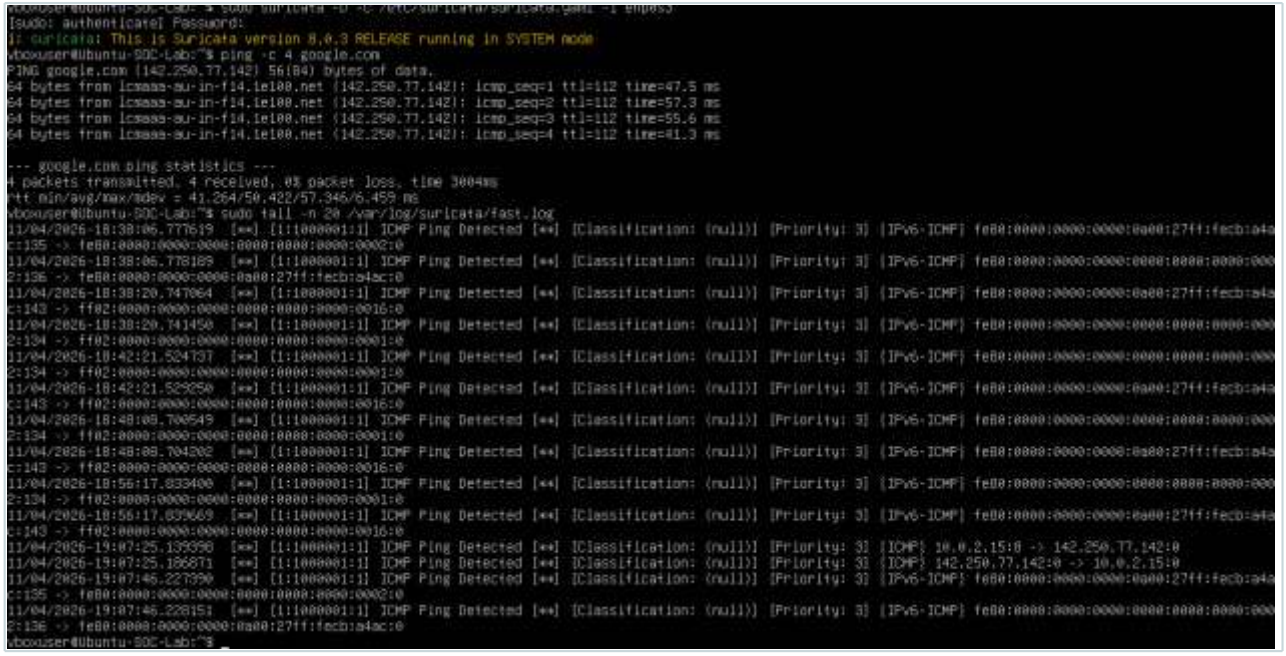


Figure 21. *Ping to google.com with 0% loss, followed by fast.log showing ICMP Ping Detected alerts (sid 1000001).*

6.3 Reading the alerts

Every line in fast.log follows the same layout. One real alert from this test, taken apart:

```
11/04/2026-19:07:25.139398  [**] [1:1000001:1] ICMP Ping Detected [**]
[Classification: (null)] [Priority: 3] {ICMP} 10.0.2.15:8 -> 142.250.77.142:0
```

Field	Value	Meaning
Timestamp	11/04/2026-19:07:25.139398	Date and time of the packet, in the VM's clock (UTC)
Rule ID	[1:1000001:1]	Generator 1, signature ID 1000001, revision 1: our ICMP rule
Message	ICMP Ping Detected	The msg text we wrote in the rule
Classification	(null)	No classtype was set in the rule, so none is shown
Priority	3	Default (lowest) priority because no priority was defined
Protocol	{ICMP}	Protocol of the packet that matched
Source and destination	10.0.2.15:8 -> 142.250.77.142:0	VM to google.com. For ICMP, the number after the colon is the message type: 8 is an echo request

The next line in the log is the reply, `142.250.77.142:0 -> 10.0.2.15:0`: source and destination are swapped and the type is 0 (echo reply). Reading source, destination and direction like this is exactly how an L1 analyst decides whether traffic was initiated from inside or outside the network.

Additional IPv6 alerts

The log also contains many alerts with `{IPv6-ICMP}` between addresses starting with `fe80::` and `ff02::`, with types such as 134, 135, 136 and 143. These were not caused by the ping test. They are routine IPv6 housekeeping messages (router advertisements, neighbour discovery and multicast listener reports) that every IPv6-enabled host produces. Because the ICMP rule matches *all* ICMP, including ICMPv6, it also reports this background chatter. This is a good real-world example of a noisy rule and of why detections need tuning.

6.4 HTTP traffic test

To trigger the second rule, an HTTP request was made from the VM with `curl http://example.com` while Suricata was running. The site returned its small HTML page, and the alert log was read again with `sudo tail -n 20 /var/log/suricata/fast.log`. Because the rule uses the `http` protocol keyword, each decoded HTTP transaction raised an alert with the message **HTTP Traffic Detected** and signature ID 1000002.

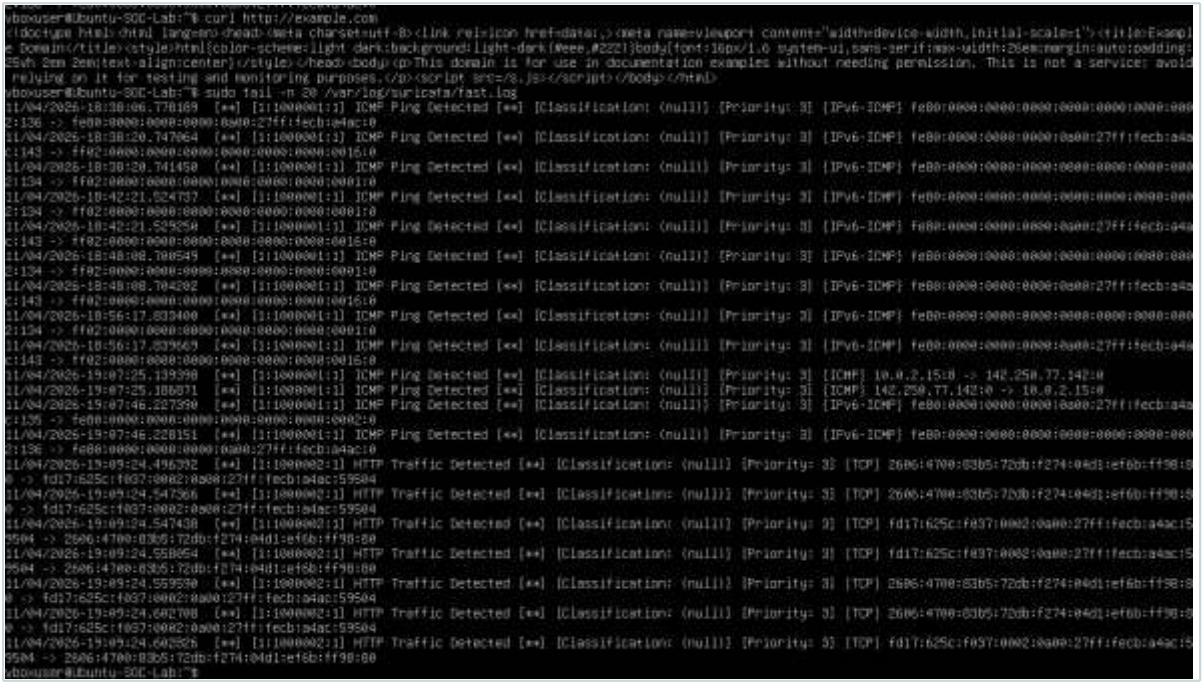


Figure 22. *fast.log showing HTTP Traffic Detected alerts (sid 1000002) after web traffic was generated.*

The new alerts are marked `{TCP}` and use IPv6 addresses (2606:4700:... and fd17:625c:...). The destination port `:80` is the web server and `:59504` is the temporary port the VM used, so the alerts show both directions of the same conversation. example.com was reached over IPv6 here, which confirms that the http rule matches by protocol rather than by IP version. Note the ICMP alerts above them in the same log: the two rules work side by side.

7. Analysis and Observations

7.1 Summary of results

Test	Rule	Result
Ping google.com (ICMP)	1000001 - ICMP Ping Detected	Detected. Echo request (type 8) and echo reply (type 0) both logged in fast.log.
Web request (HTTP)	1000002 - HTTP Traffic Detected	Detected. curl to example.com produced HTTP Traffic Detected alerts (TCP, port 80).
Background IPv6 traffic	1000001	Also detected, as ICMPv6 matches the same rule. Expected, but noisy.
Configuration test	-	Passed: configuration successfully loaded.

7.2 Problems met and how they were solved

Problem	Cause	Solution
Unable to locate package snort	Snort is not packaged for Ubuntu 26.04 in the official repositories	Verified release and sources, searched the index, then installed Suricata, a free IDS with Snort-compatible rules
No rule files match suricata.rules	No rule set was downloaded or configured by default	Wrote custom rules in /etc/suricata/rules/local.rules and re-tested
Rules appeared on one line	Missing newline between the two rules	Edited in nano so each rule has its own line
Could not execute systemctl at install	Service could not be started during package configuration	Started Suricata manually with -D and verified alerts in fast.log
25% packet loss on first ping	Brief delay on the NAT interface	Repeated the test; 0% loss, so detection testing was reliable

7.3 How a SOC analyst would use these alerts

As an L1 analyst, the alerts above would be handled in three steps. First, **validate**: is the traffic expected? A ping from 10.0.2.15 to google.com is the intern running a connectivity test, so it is benign. Second, **assess context**: which host initiated it, how often, and to where? A single ping is harmless, but hundreds of pings sweeping a subnet would suggest network reconnaissance. Third, **decide**: close as a false positive, document it, or escalate to L2. The alerts here are information-level detections that prove the monitoring works, and they show the raw material a SIEM would later correlate.

7.4 Improving the rules

The lab rules are intentionally broad. In a real environment they would be narrowed to cut noise. Examples of how this could be done (not applied in this lab):

```
alert icmp any any -> $HOME_NET any (msg:"ICMP Echo Request"; itype:8; sid:1000003; rev:1;)
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP GET request";
    flow:established,to_server; http.method; content:"GET"; sid:1000004; rev:1;)
```

- **itype:8** limits the ICMP rule to echo requests, so replies and IPv6 background messages no longer trigger it.
- **flow:established,to_server** and **http.method** restrict the HTTP rule to real client requests instead of every HTTP packet.
- Setting **classtype** and **priority** would let analysts sort alerts by severity.
- Forwarding the alerts to a SIEM such as Wazuh or Elastic would add correlation and dashboards, which is the natural next step for this lab.

8. Conclusion

This week combined theory and practice. On the theory side, a SOC was described as a combination of people, process and technology; the L1, L2 and L3 tiers were compared; and SIEM, IDS and IPS were distinguished by purpose, placement and action.

On the practical side, a complete monitoring lab was built from scratch: an Ubuntu Server VM was installed, updated, named and checked on the network; Snort turned out to be unavailable on this release, so Suricata was installed and verified instead; two custom rules were written for ICMP and HTTP; and live traffic generated alerts that were read and interpreted from the fast.log file.

Key lessons

- Always verify the tool exists for your OS release; having a Snort-compatible alternative is a practical skill.
- Test mode (`-T`) is the quickest way to find configuration and rule errors before going live.
- Rule syntax is unforgiving: one missing newline changes how the file is read.
- Broad rules prove the pipeline works but create noise; tuning is a core part of SOC work.
- An IDS alone only detects. Its real value appears when its alerts feed a SIEM and a trained analyst.

Future work

Next steps could include downloading a maintained rule set with suricata-update, switching to JSON output (eve.json) and visualising it in a SIEM, writing rules for SSH brute-force or port-scan detection, and testing inline IPS mode.

Appendix A - Commands used

```
sudo apt update && sudo apt upgrade -y          # system update
hostname                                          # verify hostname
ip a                                              # verify network
sudo apt install snort -y                         # fails: package not found
lsb_release -a ; apt search snort                 # troubleshooting
sudo apt install suricata -y                      # IDS installation
suricata --build-info ; suricata -h               # verification
sudo suricata -T -c /etc/suricata/suricata.yaml #  test mode
sudo nano /etc/suricata/suricata.yaml             # review HOME_NET / rule paths
sudo nano /etc/suricata/rules/local.rules         # create the two rules
sudo suricata -D -c /etc/suricata/suricata.yaml -i enp0s3
ping -c 4 google.com                              # ICMP test traffic
sudo tail -n 20 /var/log/suricata/fast.log        # read alerts
```

Appendix B - References

- Suricata documentation: https://docs.suricata.io
- Suricata rules format: https://docs.suricata.io/en/latest/rules/intro.html
- Ubuntu Server documentation: https://ubuntu.com/server/docs
- Oracle VirtualBox documentation: https://www.virtualbox.org/manual/